

AI Assisted Coding

Assignment 13.5

Name : R.Bharadwaj

Hall ticket no: 2303A51610

Batch no: 19

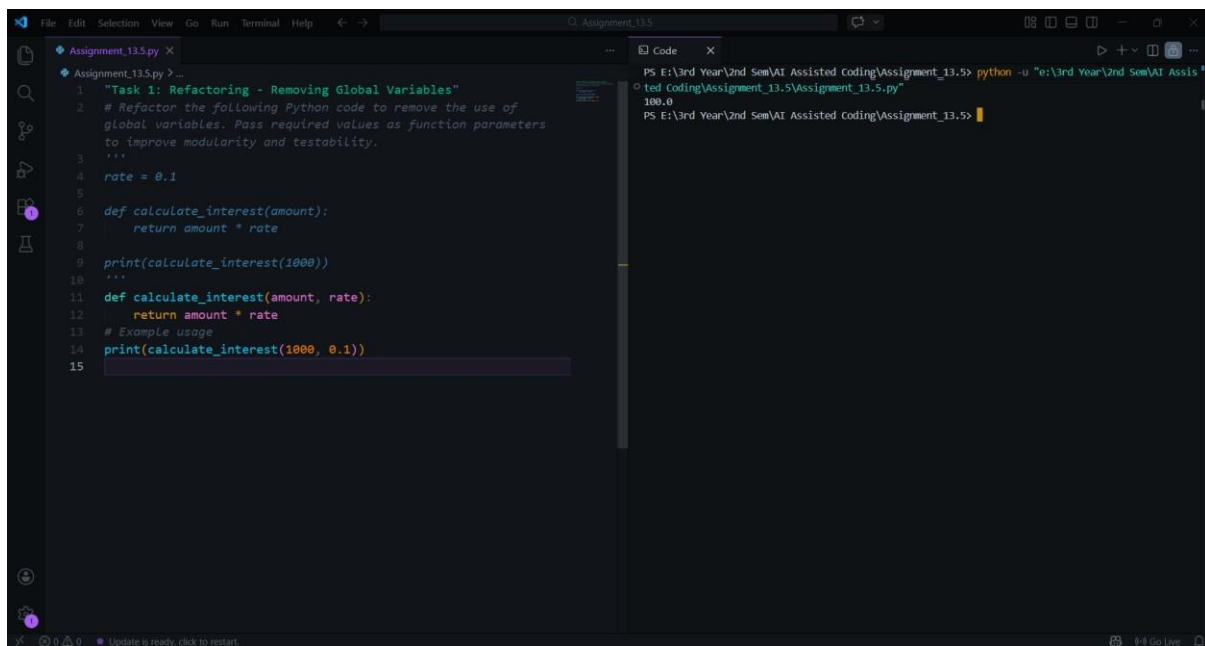
Task 1: Refactoring – Removing Global Variables

Prompt:

Refactor the following Python code to remove the use of global variables. Pass required values as function parameters to improve modularity and testability.

```
rate = 0.1
def calculate_interest(amount):
    return amount * rate
print(calculate_interest(1000))
```

Code & Output:

The screenshot shows a code editor with two panes. The left pane displays the refactored Python code for 'Assignment_13.5.py'. The code includes a docstring for 'Task 1: Refactoring - Removing Global Variables', a comment about refactoring to remove global variables, and two function definitions. The first function, 'calculate_interest(amount)', uses a global 'rate' variable. The second function, 'calculate_interest(amount, rate)', is the refactored version that takes 'rate' as a parameter. An example usage is shown at the bottom. The right pane shows the terminal output of running the code, which prints '100.0'.

Explanation:

The original code used a global variable `rate`, which reduces modularity. The refactored version passes `rate` as a function parameter, making the function independent of global state. This improves testability and reusability.

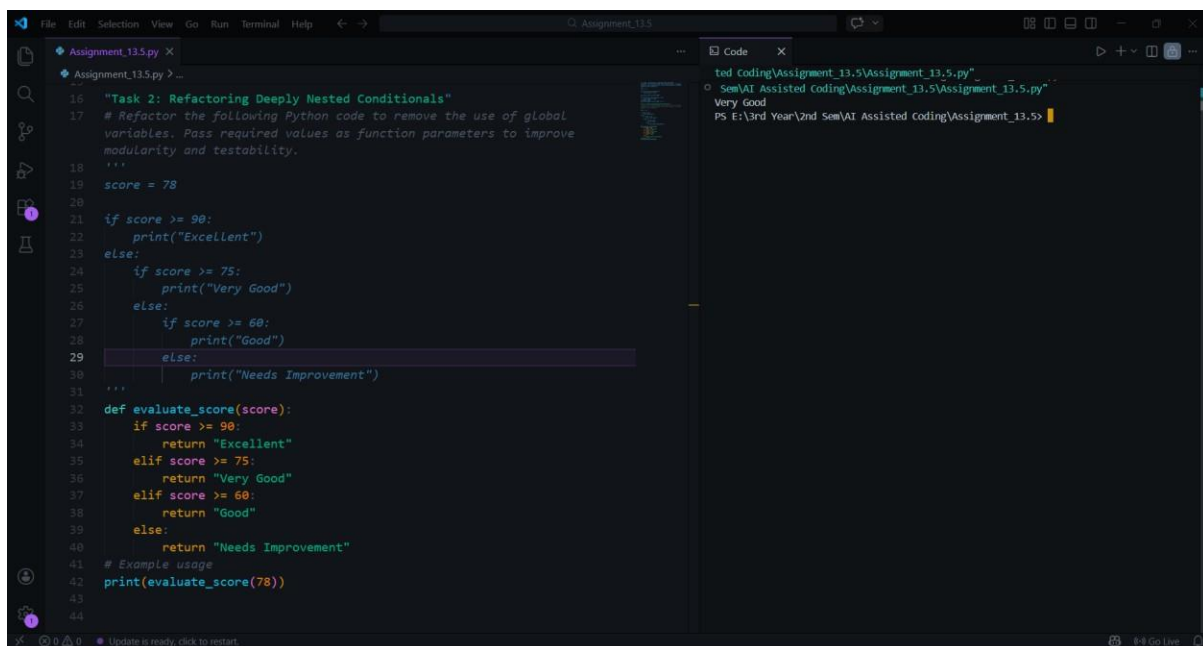
Task 2: Refactoring Deeply Nested Conditionals

Prompt:

Refactor the nested conditional statements into a cleaner and more readable structure.

```
score = 78
if score >= 90:
    print("Excellent")
else:
    if score >= 75:
        print("Very Good")
    else:
        if score >= 60:
            print("Good")
        else:
            print("Needs Improvement")
```

Code & Output:



The screenshot shows a code editor with two panels. The left panel displays the original code with deeply nested if-else statements for evaluating a score. The right panel shows the refactored code, where the logic is encapsulated into a function named `evaluate_score`. The refactored code uses a flat if-elif-else structure, which is cleaner and more readable. The output of the code is shown in the bottom right corner, displaying "Very Good" for a score of 78.

```
16 "Task 2: Refactoring Deeply Nested Conditionals"
17 # Refactor the following Python code to remove the use of global
18 # variables. Pass required values as function parameters to improve
19 # modularity and testability.
20 """
21 score = 78
22 if score >= 90:
23     print("Excellent")
24 else:
25     if score >= 75:
26         print("Very Good")
27     else:
28         if score >= 60:
29             print("Good")
30         else:
31             print("Needs Improvement")
32 """
33 def evaluate_score(score):
34     if score >= 90:
35         return "Excellent"
36     elif score >= 75:
37         return "Very Good"
38     elif score >= 60:
39         return "Good"
40     else:
41         return "Needs Improvement"
42 # Example usage
43 print(evaluate_score(78))
44
```

Explanation:

The legacy code contains multiple nested if statements, which makes the control flow harder to read. The refactored version replaces nested conditions with a flat if-elif-else structure. This simplifies the logic and improves readability. It also follows standard Python coding practices for writing clear conditional statements.

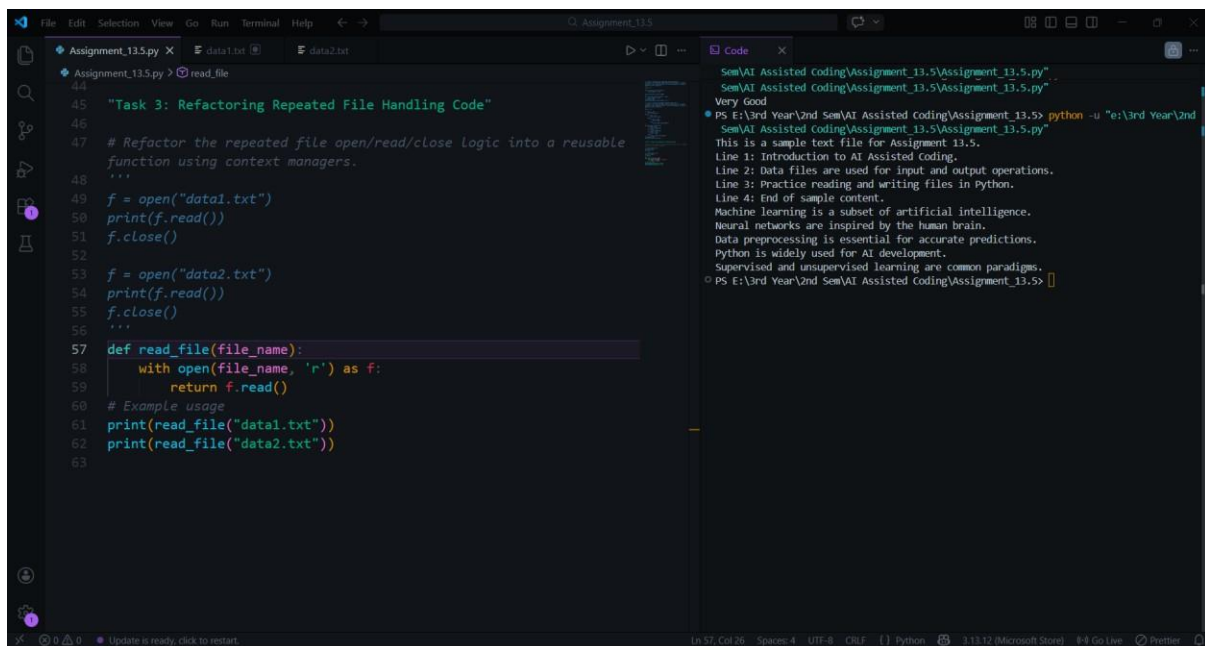
Task 3: Refactoring Repeated File Handling Code

Prompt:

Refactor the repeated file open/read/close logic into a reusable function using context managers.

```
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
```

Code & Output:

The screenshot shows a code editor with two panes. The left pane displays the Python code for 'Task 3: Refactoring Repeated File Handling Code'. The code defines a function 'read_file' that uses 'with open()' to read a file. It then calls this function for 'data1.txt' and 'data2.txt'. The right pane shows the output of the code, which includes a 'Very Good' message and the contents of the two data files. The output text is: 'Very Good', 'This is a sample text file for Assignment 13.5.', 'Line 1: Introduction to AI Assisted Coding.', 'Line 2: Data files are used for input and output operations.', 'Line 3: Practice reading and writing files in Python.', 'Line 4: End of sample content.', 'Machine learning is a subset of artificial intelligence.', 'Neural networks are inspired by the human brain.', 'Data preprocessing is essential for accurate predictions.', 'Python is widely used for AI development.', 'Supervised and unsupervised learning are common paradigms.'

Explanation:

The original code repeatedly opens and closes files, which results in duplicated code. The refactored solution introduces a reusable function and uses the with open() context manager. This ensures that the file is automatically closed after use. It improves maintainability and follows the DRY principle.

Task 4: Optimizing Search Logic

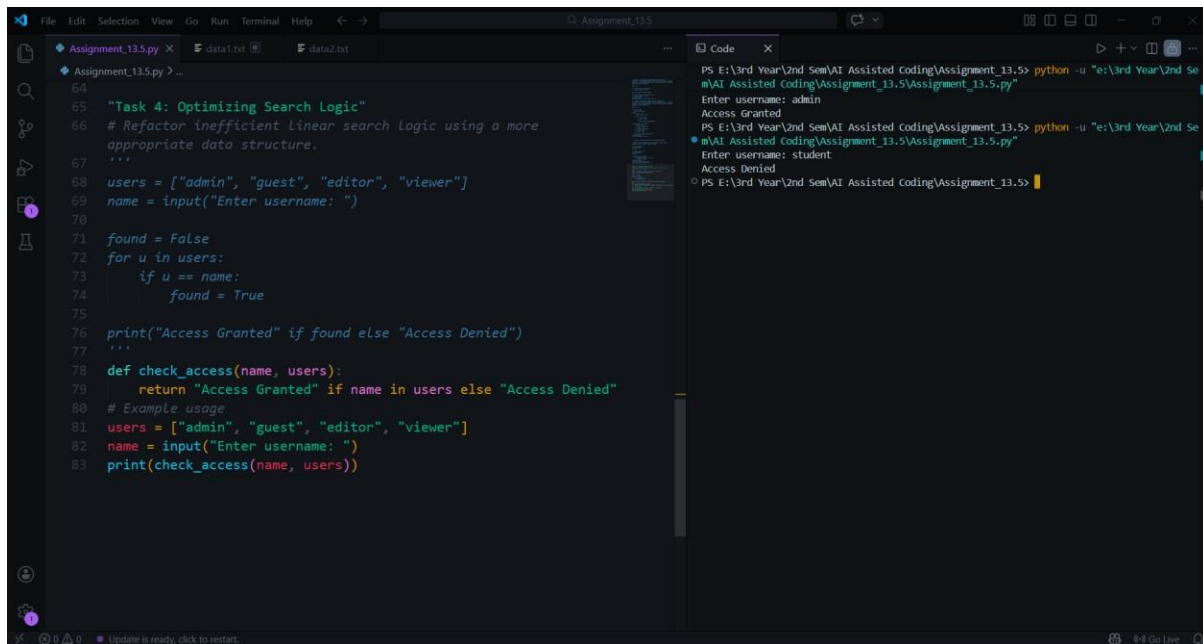
Prompt:

Refactor inefficient linear search logic using a more appropriate data structure.

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
    if u == name:
```

```
found = True
print("Access Granted" if found else "Access Denied")
```

Code & Output:



The screenshot shows a code editor with two panes. The left pane displays Python code for a user access check. The right pane shows the terminal output of the code execution.

```
64
65 "Task 4: Optimizing Search Logic"
66 # Refactor inefficient linear search logic using a more
67   appropriate data structure.
68 users = ["admin", "guest", "editor", "viewer"]
69 name = input("Enter username: ")
70
71 found = False
72 for u in users:
73     if u == name:
74         found = True
75
76 print("Access Granted" if found else "Access Denied")
77
78 def check_access(name, users):
79     return "Access Granted" if name in users else "Access Denied"
80 # Example usage
81 users = ["admin", "guest", "editor", "viewer"]
82 name = input("Enter username: ")
83 print(check_access(name, users))
```

Terminal Output:

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5> python -u "e:\3rd Year\2nd Se
m\AI Assisted Coding\Assignment_13.5\Assignment_13.5.py"
Enter username: admin
Access Granted
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5> python -u "e:\3rd Year\2nd Se
m\AI Assisted Coding\Assignment_13.5\Assignment_13.5.py"
Enter username: student
Access Denied
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5>
```

Explanation:

The legacy code uses a loop to search through a list, which requires checking each element sequentially. This results in linear time complexity. The refactored version uses a set data structure, which allows constant-time membership checking. This improves performance and makes the code simpler.

Task 5: Refactoring Procedural Code into OOP Design

Prompt:

Refactor the following procedural salary calculation code into a class-based design.

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

Code & Output:

The screenshot shows a code editor with a file named `Assignment_13.5.py`. The code is divided into two parts: a legacy procedural version and a refactored class-based version. The legacy code (lines 84-92) calculates a net salary by subtracting a 20% tax from a base salary of 50,000. The refactored code (lines 93-105) introduces a `SalaryCalculator` class with methods `__init__`, `calculate_tax`, and `calculate_net_salary`. An example usage is provided at the bottom, creating a `SalaryCalculator` instance and calling its `calculate_net_salary` method. On the right, a terminal window shows the execution of the Python script, displaying the output: `40000.0`.

```
84
85 "Task 5: Refactoring Procedural Code into OOP Design"
86 # Refactor the following procedural salary calculation code into
87   a class-based design.
88   ...
89   salary = 50000
90   tax = salary * 0.2
91   net = salary - tax
92   print(net)
93   ...
94   class SalaryCalculator:
95       def __init__(self, salary):
96           self.salary = salary
97
98       def calculate_tax(self):
99           return self.salary * 0.2
100
101       def calculate_net_salary(self):
102           tax = self.calculate_tax()
103           return self.salary - tax
104   # Example usage
105   calculator = SalaryCalculator(50000)
106   print(calculator.calculate_net_salary())
```

Explanation:

The legacy code performs calculations using simple procedural statements without structure. The refactored version introduces a class that encapsulates salary and tax calculations. This improves code organization and allows the logic to be reused easily. It also demonstrates object-oriented programming principles such as encapsulation.

Task 6: Refactoring for Performance Optimization

Prompt:

Refactor the following loop to improve performance using optimized Python techniques.

```
total = 0
for i in range(1, 1000000):
    if i % 2 == 0:
        total += i
print(total)
```

Code & Output:

```
Assignment_13.5.py
106
107 "Task 6: Refactoring for Performance Optimization"
108 # Refactor the following loop to improve performance using
    optimized Python techniques.
109 ...
110 total = 0
111 for i in range(1, 1000000):
112     if i % 2 == 0:
113         total += i
114 print(total)
115 ...
116 total = sum(i for i in range(1, 1000000) if i % 2 == 0)
117 print(total)
```

```
ted Coding\Assignment_13.5\Assignment_13.5.py"
ssisted Coding\Assignment_13.5\Assignment_13.5.py"
249999500000
PS E:\3rd Year\2nd Sem\AI Assisted coding\Assignment_13.5>
```

Explanation:

The original code manually iterates through a large range and checks each value. The refactored version uses Python's built-in `sum()` function with a generator expression. This approach reduces the number of lines and improves readability. It also takes advantage of optimized built-in functions.

Task 7: Removing Hidden Side Effects

Prompt:

Refactor the following code so that functions return values instead of modifying global variables.

```
data = []
def add_item(x):
    data.append(x)
add_item(10)
add_item(20)
print(data)
```

Code & Output:

The screenshot shows a code editor with a file named `Assignment_13.5.py`. The code is a Python script demonstrating a refactor to remove hidden side effects. It includes a function `add_item(x)` that appends to a global list `data`, and a refactored version `add_item(x, data)` that returns a new list. The output window shows the execution of the script, displaying the list `[10, 20]`.

```
Assignment_13.5.py
119 """Task 7: Removing Hidden Side Effects"""
120 # Refactor the following code so that functions return values
121 # instead of modifying global variables.
122 '''
123 data = []
124
125 def add_item(x):
126     data.append(x)
127
128 add_item(10)
129 add_item(20)
130
131 print(data)
132 '''
133 def add_item(x, data):
134     data.append(x)
135     return data
136 # Example usage
137 data = []
138 data = add_item(10, data)
139 data = add_item(20, data)
140 print(data)
```

```
ted Coding\Assignment_13.5\Assignment_13.5.py"
ssisted Coding\Assignment_13.5\Assignment_13.5.py"
249999500000
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5> python -u "E:\3rd Year\2nd Sem\AI A
ssisted Coding\Assignment_13.5\Assignment_13.5.py"
• [10, 20]
• PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5>
```

Explanation:

The legacy function modifies a global list directly, which creates hidden side effects. This can lead to unexpected behavior in larger programs. The refactored version returns a new list instead of modifying global state. This functional approach improves predictability and makes the function easier to test.

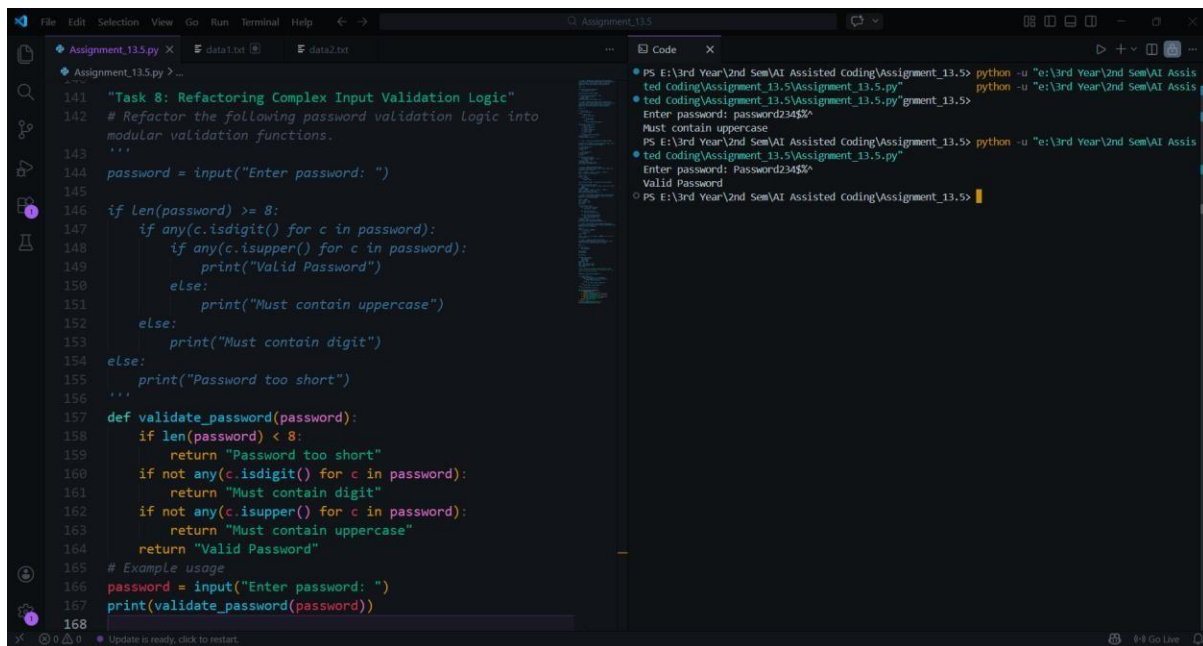
Task 8: Refactoring Complex Input Validation Logic

Prompt:

Refactor the following password validation logic into modular validation functions.

```
password = input("Enter password: ")
if len(password) >= 8:
    if any(c.isdigit() for c in password):
        if any(c.isupper() for c in password):
            print("Valid Password")
        else:
            print("Must contain uppercase")
    else:
        print("Must contain digit")
else:
    print("Password too short")
```

Code & Output:



```
141 "Task 8: Refactoring Complex Input Validation Logic"
142 # Refactor the following password validation logic into
143 # modular validation functions.
144 password = input("Enter password: ")
145
146 if len(password) >= 8:
147     if any(c.isdigit() for c in password):
148         if any(c.isupper() for c in password):
149             print("Valid Password")
150         else:
151             print("Must contain uppercase")
152     else:
153         print("Must contain digit")
154 else:
155     print("Password too short")
156 ...
157 def validate_password(password):
158     if len(password) < 8:
159         return "Password too short"
160     if not any(c.isdigit() for c in password):
161         return "Must contain digit"
162     if not any(c.isupper() for c in password):
163         return "Must contain uppercase"
164     return "Valid Password"
165
166 # Example usage
167 password = input("Enter password: ")
168 print(validate_password(password))
```

Terminal Output:

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5> python -u "e:\3rd Year\2nd Sem\AI Assis
ted Coding\Assignment_13.5\Assignment_13.5.py"
python -u "e:\3rd Year\2nd Sem\AI Assis
ted Coding\Assignment_13.5\Assignment_13.5.py"
Enter password: password234567
Must contain uppercase
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5> python -u "e:\3rd Year\2nd Sem\AI Assis
ted Coding\Assignment_13.5\Assignment_13.5.py"
Enter password: Password234567
Valid Password
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5>
```

Explanation:

The original code contains multiple nested checks that make the logic harder to read. The refactored version separates each validation rule into its own function. This modular structure improves readability and makes testing each rule easier. It also allows new validation rules to be added without modifying the entire structure.

Final Conclusion:

This lab demonstrated how AI tools can assist in refactoring legacy Python code to improve readability, modularity, and performance. Techniques such as removing global variables, simplifying nested conditions, optimizing search logic, and applying object-oriented design were used. These improvements make the code easier to understand, maintain, and extend while preserving its original functionality.